

こどもプログラミング教室 アドバンスコース5-2

ワープメイズ 改造後の解答例プログラム

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<script type="text/javascript" src="parts.js"></script>
<script type="text/javascript" src="mapc.js"></script><!--【C-2:変更】-->
<script type="text/javascript">

//ゲームの開始の時にやること
function start() {
  for (let j = 2; j < 9; j += 2) { //【C-2: 以下追加】2,4,6,8行についてくり返す
    for (let i = 2; i < 9; i += 2) { //2,4,6,8列についてくり返す
      while (true) { //決まるまでくり返す
        let d //中心から見てどの向きに新たに壁を置くか(0=右、1=左、2=下、3=上)
        if (j === 2) { //2行目ならば
          d = Math.floor(Math.random() * 4) //壁にする候補を右左下上の4通りから選ぶ
        } else { //4,6,8行目ならば
          d = Math.floor(Math.random() * 3) //壁にする候補を右左下の3通りから選ぶ
        }
        if (d === 0 && map1[j][i + 1] === 0) { //右が候補で右が道ならば
          map1[j][i + 1] = 8 //右を壁にする
          map0[j][i + 1] = 8 //右を壁にする(やり直し用)
          break //決定したので次へ
        } else if (d === 1 && map1[j][i - 1] === 0) { //左が候補で左が道ならば
          map1[j][i - 1] = 8 //左を壁にする
          map0[j][i - 1] = 8 //左を壁にする(やり直し用)
          break //決定したので次へ
        } else if (d === 2 && map1[j + 1][i] === 0) { //下が候補で下が道ならば
          map1[j + 1][i] = 8 //下を壁にする
          map0[j + 1][i] = 8 //下を壁にする(やり直し用)
          break //決定したので次へ
        } else if (d === 3 && map1[j - 1][i] === 0) { //上が候補で上が道ならば
          map1[j - 1][i] = 8 //上を壁にする
          map0[j - 1][i] = 8 //上を壁にする(やり直し用)
          break //決定したので次へ
        }
      }
    }
  }
  hidariwarp.x = hidariwarp.x + 2 * Math.floor(Math.random() * 2) //左ワープの位置を乱数で決める
  migiwarp.x = migiwarp.x - 2 * Math.floor(Math.random() * 2) //右ワープの位置を乱数で決める
  for (let i = 0; i < 6; i++) { //【B-2: 以下追加】全ての効果音についてくり返す
    oto[i] = new Audio("oto" + i + ".mp3") //ファイルを開く
    oto[i].preload = "auto" //音が自動で流れるようにセットする
    oto[i].load() //効果音を読み込んでおく
  }
  timerno = setInterval(mainloop, timer) //タイマーを起動する
}
```

改造A

改造B

改造C

```
//クリックされた時にやること
function onclick_canvas(evt) {
    mouse = mousezahyo(canvas, evt) //マウスの座標を得る
    if (mode >= 5 && !onhidariwarp() && !onmigiwarp() && onplayer()) { //【C-1: 以下追加】プレイモード以降でワープ以外でプレイヤーをクリック?
        retry() // やりなおし確認
        return //以降は実行しない
    }
}

if (mode === 0) { //タイトル画面?
    mode = 4 //準備モードにする
} else if (mode === 4 && onplayer()) { //準備モードでプレイヤーをクリック?
    oto[0].currentTime = 0 //【B-2:追加】効果音の準備
    oto[0].play() //【B-2:追加】効果音を流す
    //goalmode = 1 //【A-1:削除】ゴールを有効にする
    mode = 5 //プレイモードにする
} else if (mode === 5) { //プレイモードならば
    if (onhidariwarp()) { //左ワープをクリック?
        // player.x = migiwarp.x //【B-1: 削除】右ワープに移動
        // player.y = migiwarp.y //【B-1: 削除】右ワープに移動
        oto[2].currentTime = 0 //【B-2:追加】効果音の準備
        oto[2].play() //【B-2:追加】効果音を流す
        mode = 6 //ワープモードにする
        warpcount = warpstart //【B-1: 追加】ワープカウンタ
        warpmode = 1 //左ワープ中
    } else if (onmigiwarp()) { //右ワープをクリック?
        // player.x = hidariwarp.x //【B-1: 削除】左ワープに移動
        // player.y = hidariwarp.y //【B-1: 削除】左ワープに移動
        oto[2].currentTime = 0 //【B-2:追加】効果音の準備
        oto[2].play() //【B-2:追加】効果音を流す
        mode = 6 //ワープモードにする
        warpcount = warpstart //【B-1: 追加】ワープカウンタ
        warpmode = 2 //右ワープ中
    }
} else if (mode === 6 && onplayer()) { //ワープモードでプレイヤーをクリック?
    mode = 5 //プレイモードに戻る
    warpmode = 0 //ワープ中解除
}
}
```

```
//マウスが動いた時にやること
function onmove_canvas(evt) {
  if (mode === 5) { //プレイモード?
    mouse = mousezahyo(canvas, evt) //マウスの座標を得る
    playerp.x = player.x //現在のプレイヤーの位置xを保存
    playerp.y = player.y //現在のプレイヤーの位置yを保存
    playerdousa() //プレイヤーの動き
    if (hcenter() && vcenter()) { //プレイヤーが上下左右共にマスの真ん中にいれば
      tuukashori(200, 10) //初めての通過場所ならスコアを200加点して通過済にし、通過済の場所なら10減点する
      if (fruitari === 1 && player.x === fruit.x && player.y === fruit.y) { //【A-1: 以下追加】フルーツの上なら
        oto[1].currentTime = 0 //【B-2:追加】効果音の準備
        oto[1].play() //【B-2:追加】効果音を流す
        fruitari = 0 //フルーツを消す
        goalmode = 1 //ゴールを有効にする
      }
    }
    if (goalmode === 1 && map1[player.y][player.x] === 3) { //ゴールなら
      hiscoreupdate() //【C-1:追加】ハイスコアの更新
      mode = 9 //クリアモードへ
      if (allclear() === 1) { //【B-2:以下追加】完全クリアであれば
        oto[5].currentTime = 0 //効果音の準備
        oto[5].play() //効果音を流す
      } else {
        oto[4].currentTime = 0 //効果音の準備
        oto[4].play() //効果音を流す
      }
    }
  }
}

if (playerp.x !== player.x || playerp.y !== player.y) { //プレイヤーが動いたら
  tekidousa(25) //【C-2: 変更】敵も動く(0: 動かない~100: 最高速)
}

if (hantei()) { //プレイヤーが敵にさわられたら
  oto[3].currentTime = 0 //【B-2:追加】効果音の準備
  oto[3].play() //【B-2:追加】効果音を流す
  hiscoreupdate() //【C-1:追加】ハイスコアの更新
  mode = 8 //ゲームオーバー状態へ
}
}
}

//一定時間おきにやること
function mainloop() {
  if (mode === 6 && warpcount > -warpstart) { //【B-1:以下追加】ワープモードならば
    warpcount-- //ワープカウンタを減らす
    if (warpcount < 0) { //移動先にワープする?
      if (warpmode === 1) { //左ワープからならば
        player.x = migiwarp.x //右ワープに移動
        player.y = migiwarp.y //右ワープに移動
      } else { //右ワープからならば
        player.x = hidariwarp.x //左ワープに移動
        player.y = hidariwarp.y //左ワープに移動
      }
    }
  }
  draw() //画面をえがく処理を呼ぶ
}
```

```
//迷路などをえがく
function drawmap() {
  fillblack(24, 24, 432, 432) //迷路部分を黒くぬりつぶす
  for (let j = 0; j < 11; j++) { //迷路全行についてくり返す
    for (let i = 0; i < 11; i++) { //迷路全列についてくり返す
      if (map1[j][i] === 8) { //内壁ならば
        ctx.drawImage(wall, i * 48 - 24, j * 48 - 24) //壁を表示
      } else if (map1[j][i] === 3) { //ゴールならば
        ctx.drawImage(goal[goalmode], i * 48 - 24, j * 48 - 24) //ゴール[goalmode]を表示
      } else if (map1[j][i] === 1) { //通過済ならば
        fillblue48(i, j) //48×48を青くぬりつぶす
      }
    }
  }

  ctx.drawImage(warp, hidariwarp.x * 48 - 24, hidariwarp.y * 48 - 24) //左ワープを表示
  ctx.drawImage(warp, migiwarp.x * 48 - 24, migiwarp.y * 48 - 24) //右ワープを表示
  if (fruitari === 1) { //【A-1: 以下追加】フルーツがあれば
    ctx.drawImage(fruiting, 216, 24) //フルーツを表示
  }

  if (mode === 6 && warpcount > -warpstart) { //【B-1:追加】ワープモードでアニメーション中ならば
    ctx.drawImage(playerimg, player.x * 48 - 24, player.y * 48 - 24, 48, Math.abs(warpcount)) //【B-1:追加】プレイヤーを縮小表示
  } else { //【B-1:追加】でなければ
    ctx.drawImage(playerimg, player.x * 48 - 24, player.y * 48 - 24) //プレイヤーを表示
  } //【B-1:追加】
  ctx.drawImage(enemy[tekimuki], teki.x * 48 - 24, teki.y * 48 - 24) //敵を表示
  drawscore(score) //スコアボードを表示してその上に白でスコアを6桁でえがく
  ctx.drawImage(hiscoreboard, 478, 322) //【C-1: 追加】ハイスコアボードを表示
  ctx.font = '20pt Arial' //【C-1: 追加】文字サイズを指定
  for (let i = 0; i < 3; i++) { //【C-1: 以下追加】ハイスコアを表示
    ctx.fillText((i + 1) + ":" + "000000" + hiscore[i]).slice(-6), 490, 380 + 30 * i) //ハイスコア[0~2]を6桁で表示
  }
}

//画面をえがく
function draw() {
  if (mode === 0) { //タイトル画面表示ならば
    fillblack(0, 0, 640, 480) //背景を黒くぬりつぶす
    let xs = (window1.sx - window1.sy) / 2 //開始X座標を得る
    for (let j = 0; j < 5; j++) { //5行についてくり返す
      for (let i = 0; i < 5; i++) { //5列についてくり返す
        if (i !== 2) { //中央列でなければ
          ctx.drawImage(wall, xs + 96 * i, 96 * j, 96, 96) //壁画像を表示
        }
      }
    }
    ctx.drawImage(title, xs + 96, 96) //タイトル画像を表示
  } else if (mode === 4) { //準備モードならば
    haikewall() //画面全体に壁をえがきミニタイトルを表示する
    drawmap() //迷路などをえがく
    drawmessage(0) //メッセージを表示
  } else if (mode === 5) { //プレイモードならば
    drawmap() //迷路などをえがく
    if ((onhidariwarp() || onmigiwarp()) && onplayer()) { //プレイヤーがワープの上にいれば
      drawmessage(5) //メッセージを表示
    } else if (fruitari === 1) { //【A-1: 追加】フルーツがあれば
      drawmessage(7) //【A-1: 追加】メッセージを表示
    } else {
      drawmessage(1) //メッセージを表示
    }
  } else if (mode === 6) { //ワープ状態ならば
    drawmap() //迷路などをえがく
    drawmessage(6) //メッセージを表示
  } else if (mode === 8) { //ゲームオーバー状態ならば
    drawmap() //迷路などをえがく
    drawmessage(4) //メッセージを表示
    drawcenter("Game Over", 18, 64) //「Game Over」表示
  } else if (mode === 9) { //クリア状態ならば
    drawmap() //迷路などをえがく
    if (allclear() === 1) { //完全クリアであれば
      drawmessage(2) //メッセージを表示
      drawcenter("CLEAR!", 18, 96) //「CLEAR!」表示
    } else { //完全クリアでなければ
      drawmessage(3) //メッセージを表示
      drawcenter("GOAL!", 48, 96) //「GOAL!」表示
    }
  }
}
}
```

```
</script>
```

```
</head>
```

```
<body onload="init();" > <!-- 最初に1回だけやること(起動処理)を示す -->
<canvas id="canvas" width="640" height="480" /></canvas> <!-- キャンバスの設置 -->
 <!-- 壁画像おき場 -->
 <!-- タイトル画像おき場 -->
 <!-- ミニタイトル画像おき場 -->
 <!-- プレイヤー画像おき場 -->
 <!-- ゴール画像[0]おき場 -->
 <!-- ゴール画像[1]おき場 -->
 <!-- ワープ画像おき場 -->
 <!-- メッセージ画像[0]おき場 -->
 <!-- メッセージ画像[1]おき場 -->
 <!-- メッセージ画像[2]おき場 -->
 <!-- メッセージ画像[3]おき場 -->
 <!-- メッセージ画像[4]おき場 -->
 <!-- メッセージ画像[5]おき場 -->
 <!-- メッセージ画像[6]おき場 -->
 <!--【A-1:追加】メッセージ画像[7]おき場 -->
 <!-- スコアボード画像おき場 -->
 <!-- 敵画像[0]おき場 -->
 <!-- 敵画像[1]おき場 -->
 <!--【A-1:追加】フルーツ画像おき場 -->
 <!--【C-1:追加】ハイスコアボード画像おき場 -->
</body>
</html>
```